

Содержание

1. Цель работы	3
2. Теоретическое введение	3
2.1. Почему CNN, а не MLP	3
2.2. Свёртка	3
2.3. Pooling	4
2.4. Нелинейность ReLU	4
2.5. Архитектура сети	4
2.6. Функция потерь	5
2.7. Оптимизация и Adam	5
2.8. Ранняя остановка	5
2.9. Train / Val / Test	5
3. Программная реализация	5
3.1. Структура проекта	5
3.2. Свёрточные блоки	6
3.3. Класс ImageCNN	6
3.4. Цикл обучения	7
3.5. Метрики	7
3.5.1. Accuracy	7
3.5.2. Confusion matrix	7
4. Часть I. Эксперимент на MNIST	7
4.1. Описание выборок и параметров	7
4.2. Превью датасета	8
4.3. Сетка конфигураций	8
4.4. Топ конфигураций по val_acc	9
4.5. Лучшая конфигурация	9
4.6. Графики и их интерпретация	10
4.6.1. Сводный график конфигураций	10
4.6.2. Кривые обучения лучшей модели	10
4.6.3. Confusion matrix на test	11
4.6.4. Влияние отдельных гиперпараметров	12
4.7. Блок 2: оптимизаторы	14
4.8. Блок 2: переобучение и Dropout	15
5. Часть II. Эксперимент на FashionMNIST	15
5.1. Превью датасета	16
5.2. Параметры эксперимента	16
5.3. Топ конфигураций по val_acc	17
5.4. Лучшая конфигурация	17

5.5. Графики и их интерпретация	18
5.6. Блок 2: оптимизаторы	22
5.7. Блок 2: переобучение и Dropout	22
6. Сравнение MNIST и FashionMNIST	23
7. Выводы	24
8. Структура проекта	24
9. Репозиторий	25

1. Цель работы

Цель лабораторной работы — реализация свёрточной нейронной сети (CNN, Convolutional Neural Network) средствами библиотеки `torch.nn` и исследование её возможностей при классификации изображений.

В работе исследуются:

- архитектуры CNN с двумя типами свёрточных блоков (A и B);
- влияние гиперпараметров: число каналов, размер ядра свёртки, шаг pooling;
- сравнение оптимизаторов (Adam, AdamW, SGD) и learning rate;
- демонстрация переобучения и эффекта Dropout;
- финальное обучение лучшей модели на полной обучающей выборке;
- эксперимент проводится на двух датасетах: **MNIST** (рукописные цифры) — основной, и **FashionMNIST** (предметы одежды) — сравнительный.

2. Теоретическое введение

2.1. Почему CNN, а не MLP

Для изображений размера 28×28 при подаче в обычный многослойный перцептрон вход разворачивается в вектор длины 784, из-за чего теряется информация о пространственной структуре. CNN использует две ключевые идеи:

- **локальные receptive fields** — нейрон смотрит не на всё изображение целиком, а на небольшую окрестность пикселя;
- **shared weights** — один и тот же фильтр применяется ко всем участкам изображения, поэтому сеть учится распознавать один и тот же тип признака в разных местах.

За счёт этого CNN сохраняет геометрию изображения, требует меньше параметров и устойчивее обучается на задачах компьютерного зрения.

2.2. Свёртка

Свёрточный слой `Conv2d` применяет к входному изображению набор обучаемых фильтров. Каждый фильтр — маленькое ядро (например, 3×3 или 5×5), скользящее по изображению:

$$y(i, j) = \sum_u \sum_v x(i + u, j + v) K(u, v) + b.$$

Один фильтр может научиться реагировать на вертикальные границы, другой — на дуги, третий — на штрихи. Если фильтров несколько, на выходе слоя получается набор каналов признаков.

Размеры на выходе свёрточного слоя по одной координате:

$$\text{out} = \left\lfloor \frac{\text{in} + 2p - k}{s} + 1 \right\rfloor,$$

где p — padding, k — размер ядра, s — stride.

2.3. Pooling

После свёртки в каждом блоке используется `MaxPool2d`. Эта операция:

- уменьшает пространственный размер карт признаков;
- оставляет наиболее сильные локальные отклики;
- даёт устойчивость к небольшим сдвигам объектов;
- снижает вычислительную нагрузку на следующие слои.

2.4. Нелинейность ReLU

После каждой свёртки применяется функция активации ReLU:

$$\text{ReLU}(z) = \max(0, z).$$

Положительные отклики пропускаются, отрицательные — зануляются. Без нелинейности несколько свёрток подряд были бы эквивалентны одной линейной операции.

2.5. Архитектура сети

Финальная архитектура `ImageCNN`:

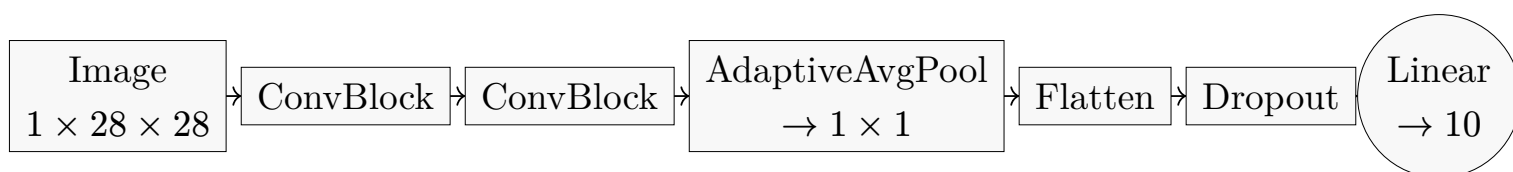


Рис. 1. Общая структура `ImageCNN`.

В лабораторной используются два варианта свёрточного блока:

ConvBlockA

Conv -> ReLU -> MaxPool

Меньше параметров, быстрее

ConvBlockB

Conv -> ReLU -> Conv -> ReLU -> MaxPool

Глубже, сложнее признаки

После всех свёрток выполняется `AdaptiveAvgPool2d((1, 1))`, что сводит каждую карту признаков к одному числу. Это делает модель универсальной по форме входа и устраняет необходимость вручную вычислять размер тензора перед `Linear`.

2.6. Функция потерь

На выходе сеть выдаёт вектор логитов $z \in \mathbb{R}^{10}$ для 10 классов. Используется `CrossEntropyLoss`, объединяющая `log_softmax` и отрицательное логарифмирование:

$$L = -\log p_y, \quad p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}.$$

2.7. Оптимизация и Adam

Параметры обновляются методом Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$w_{t+1} = w_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},$$

где $g_t = \nabla L(w_t)$, $\hat{m}_t$ и $\hat{v}_t$ — коррекция начального смещения. В работе также сравниваются AdamW (decoupled weight decay) и SGD с momentum.

2.8. Ранняя остановка

В функции `train_model(...)` реализована ранняя остановка по `val_acc`. Если за PATIENCE эпох улучшение меньше MIN_DELTA, обучение прерывается, и загружается лучшее сохранённое состояние. Это защищает от лишних эпох после плато и от ухода в переобучение.

2.9. Train / Val / Test

В работе используется три выборки:

- **train** — по ней считаются градиенты и обновляются веса;
- **val** — по ней выбирается лучшая архитектура;
- **test** — используется только для финальной честной оценки.

Если выбирать модель сразу по **test**, оценка становится смещённой — именно поэтому val-выборка существует отдельно.

3. Программная реализация

3.1. Структура проекта

Файл	Назначение
------	------------

config.py	Конфигурация: датасет, размер выборок, эпохи, early stopping, параметры DataLoader.
data.py	Загрузка MNIST / FashionMNIST, нормализация, подвыборки, разбиение на train/val, DataLoader.
cnn.py	Свёрточные блоки, класс ImageCNN, фабрика оптимизаторов, цикл обучения.
block1_main.py	Перебор гиперпараметров, выбор лучшей модели, дообучение на полном train.
block2_main.py	Сравнение оптимизаторов и эксперимент по переобучению с Dropout.

3.2. Свёрточные блоки

```
class ConvBlockA(nn.Module):
    def __init__(self, in_c, out_c, k, s, pk, ps):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_c, out_c, k, s, padding=k // 2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(pk, ps),
        )

class ConvBlockB(nn.Module):
    def __init__(self, in_c, out_c, k, s, pk, ps):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_c, out_c, k, s, padding=k // 2),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, k, 1, padding=k // 2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(pk, ps),
        )
```

3.3. Класс ImageCNN

```
class ImageCNN(nn.Module):
    def __init__(self, in_c, num_classes, channels, block_type,
                  conv_k, conv_s, pool_k, pool_s, dropout):
        super().__init__()
        Block = ConvBlockA if block_type == 'a' else ConvBlockB
        layers, prev = [], in_c
        for c in channels:
            layers.append(Block(prev, c, conv_k, conv_s, pool_k, pool_s))
            prev = c
        self.features = nn.Sequential(*layers)
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
```

```

nn.Flatten(),
nn.Dropout(dropout),
nn.Linear(prev, num_classes),
)

```

3.4. Цикл обучения

```

def train_model(model, train_loader, val_loader, optimizer,
                loss_fn, epochs, patience, min_delta):
    best_val_acc, bad = 0.0, 0
    best_state = copy.deepcopy(model.state_dict())
    history = {'train_loss': [], 'val_loss': [],
               'train_acc': [], 'val_acc': []}
    for ep in range(epochs):
        tl, ta = train_one_epoch(model, train_loader, optimizer, loss_fn)
        vl, va = evaluate(model, val_loader, loss_fn)
        history['train_loss'].append(tl); history['train_acc'].append(ta)
        history['val_loss'].append(vl); history['val_acc'].append(va)
        if va - best_val_acc > min_delta:
            best_val_acc, bad = va, 0
            best_state = copy.deepcopy(model.state_dict())
        else:
            bad += 1
            if bad >= patience: break
    model.load_state_dict(best_state)
    return history, best_val_acc

```

3.5. Метрики

3.5.1. Accuracy

$$\text{accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{y}_i = y_i].$$

3.5.2. Confusion matrix

Квадратная матрица $C \in \mathbb{R}^{10 \times 10}$, где C_{ij} — число объектов класса i , отнесённых сетью к классу j . Диагональ — правильные предсказания, остальное — ошибки.

4. Часть I. Эксперимент на MNIST

4.1. Описание выборок и параметров

Параметры основного эксперимента:

- датасет: **MNIST** (10 классов рукописных цифр 28×28);

- train-подвыборка: 8000 объектов, разбиение train/val: 6000/2000;
- test-подвыборка: 2000 объектов;
- batch size: 64, оптимизатор Adam, $lr = 0.001$;
- epochs ≤ 20 , ранняя остановка по val_acc.

4.2. Превью датасета

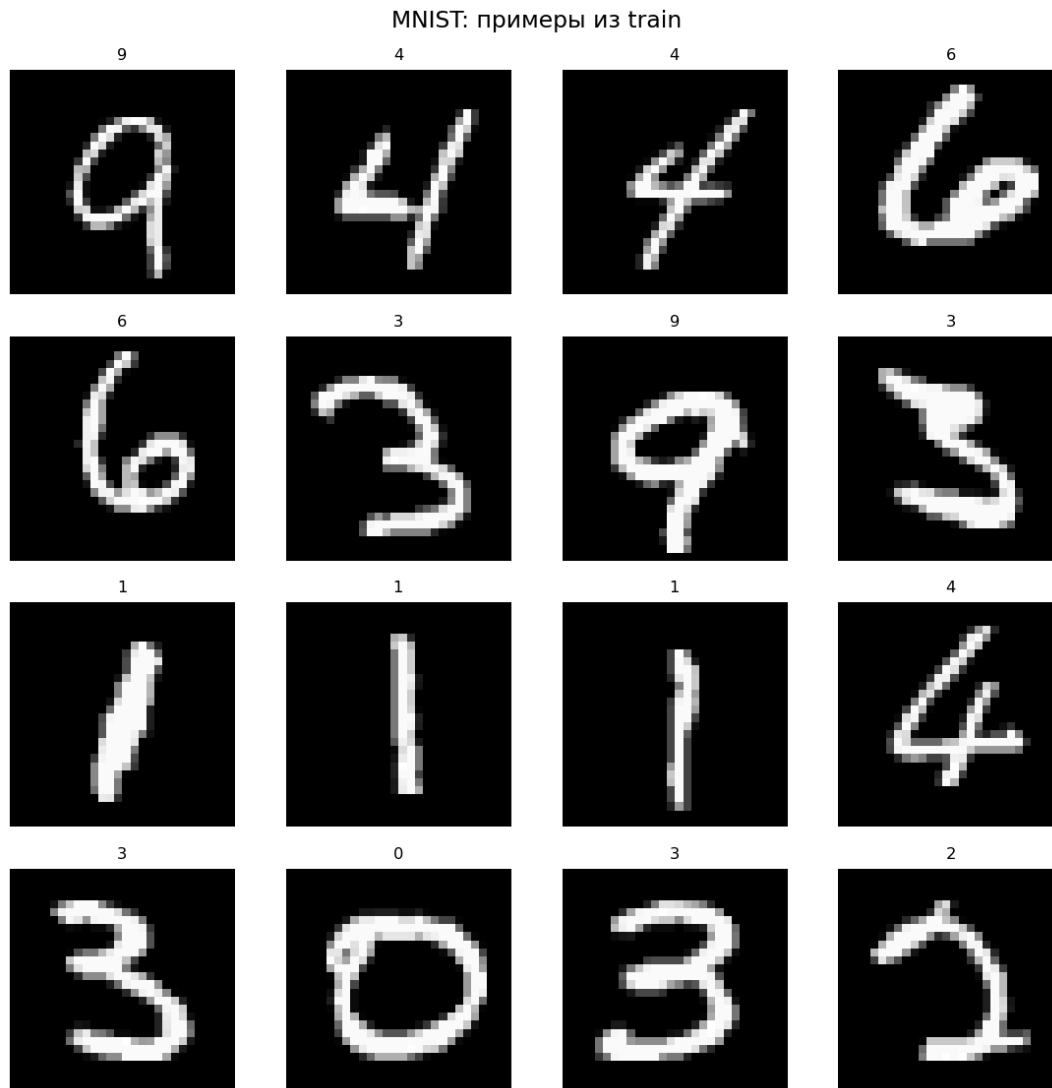


Рис. 2. Случайные изображения из обучающей выборки MNIST. Чёрно-белые рукописные цифры 28×28 с метками классов 0–9.

На картинке хорошо видны разные начертания одних и тех же цифр, сдвиги и наклоны — именно поэтому CNN с её локальными фильтрами и устойчивостью к сдвигам подходит для этой задачи лучше, чем полносвязный MLP.

4.3. Сетка конфигураций

Перебираются комбинации параметров:

- тип блока: A или B;
- число каналов: (8, 16), (16, 32), (32, 64);
- размер ядра свёртки: 3, 5, 7;

- шаг свёртки: 1, 2;
- шаг pooling: 1, 2;
- ядро pooling: 2;
- learning rate: 0.001.

Всего — 72 валидные конфигурации.

4.4. Топ конфигураций по val_acc

#	блок	каналы	ck	cs	ps	best val_acc	ep@90
70	B	(32, 64)	7	1	2	0.9875	2
71	B	(32, 64)	7	2	1	0.9870	2
58	B	(16, 32)	7	1	2	0.9855	2
59	B	(16, 32)	7	2	1	0.9845	2
67	B	(32, 64)	7	1	2	0.9825	2
68	B	(32, 64)	5	2	2	0.9815	2
36	A	(32, 64)	7	2	2	0.9810	2
60	B	(16, 32)	7	2	2	0.9805	2

Таблица 1. Топ-8 конфигураций для MNIST по best val_acc.

4.5. Лучшая конфигурация

```

block=B | channels=(32, 64) | conv(k=7, s=1) | pool(k=2, s=2)
best val_acc:    0.9875          лучшая эпоха:    15
эпох до 90%:    2              параметров:    353 642
test loss:      0.0575          test accuracy:  0.9835

```

После финального дообучения этой конфигурации на полном train (60 000 объектов) на тесте получено:

```

final test loss:    0.0251
final test accuracy: 0.9931

```

4.6. Графики и их интерпретация

4.6.1. Сводный график конфигураций

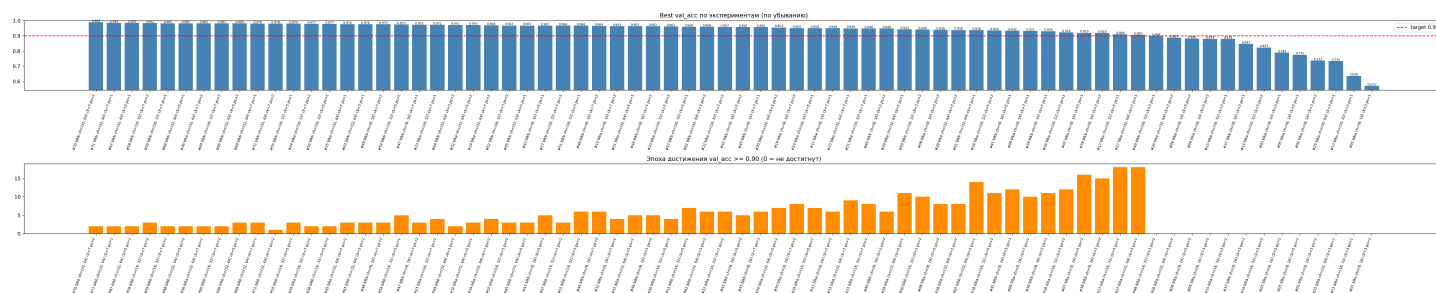


Рис. 3. Слева: `best val_acc` для всех 72 конфигураций (по убыванию). Справа: эпоха достижения `val_acc >= 90%`.

На левой панели видно, что большинство конфигураций уверенно выходит за `val_acc = 0.95`, а лидеры — блок В с (32, 64) и крупным ядром — достигают почти 0.99. Правая панель показывает, что **лучшие конфигурации достигают цели `acc ≥ 90%` уже за 2 эпохи**, тогда как слабые модели (маленькие каналы и `stride=1`) не пересекают порог даже за 20 эпох.

4.6.2. Кривые обучения лучшей модели

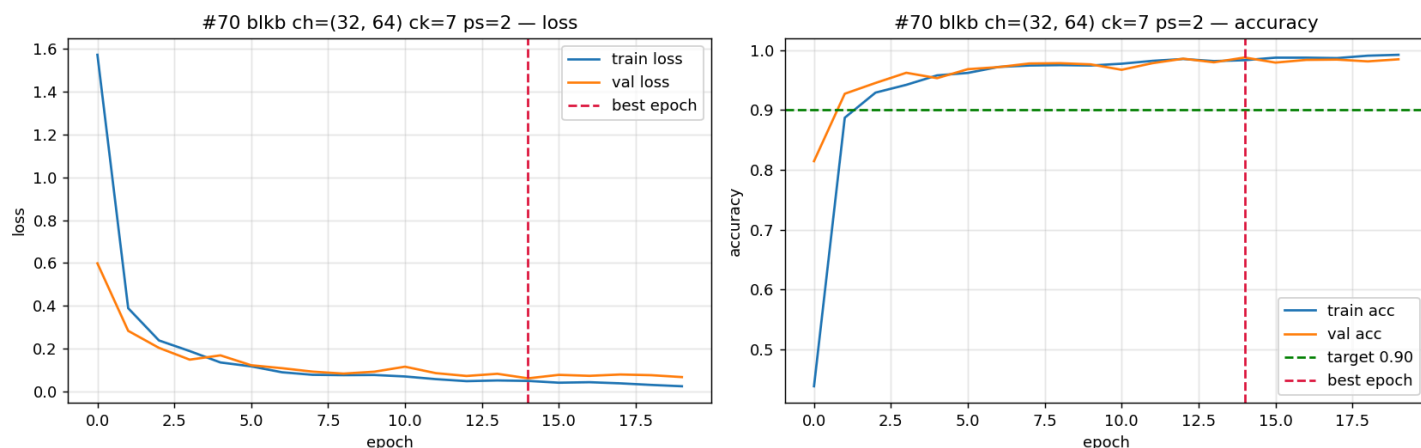


Рис. 4. Train/val loss (слева) и ассурасу (справа) для лучшей конфигурации В (32, 64) `ck=7 ps=2`.

Train и val кривые идут практически параллельно: разрыва между ними почти нет, что говорит об отсутствии переобучения. `val_acc` уже к 5–6 эпохе превышает 0.97 и далее аккуратно доходит до 0.987 на 15-й эпохе.

4.6.3. Confusion matrix на test

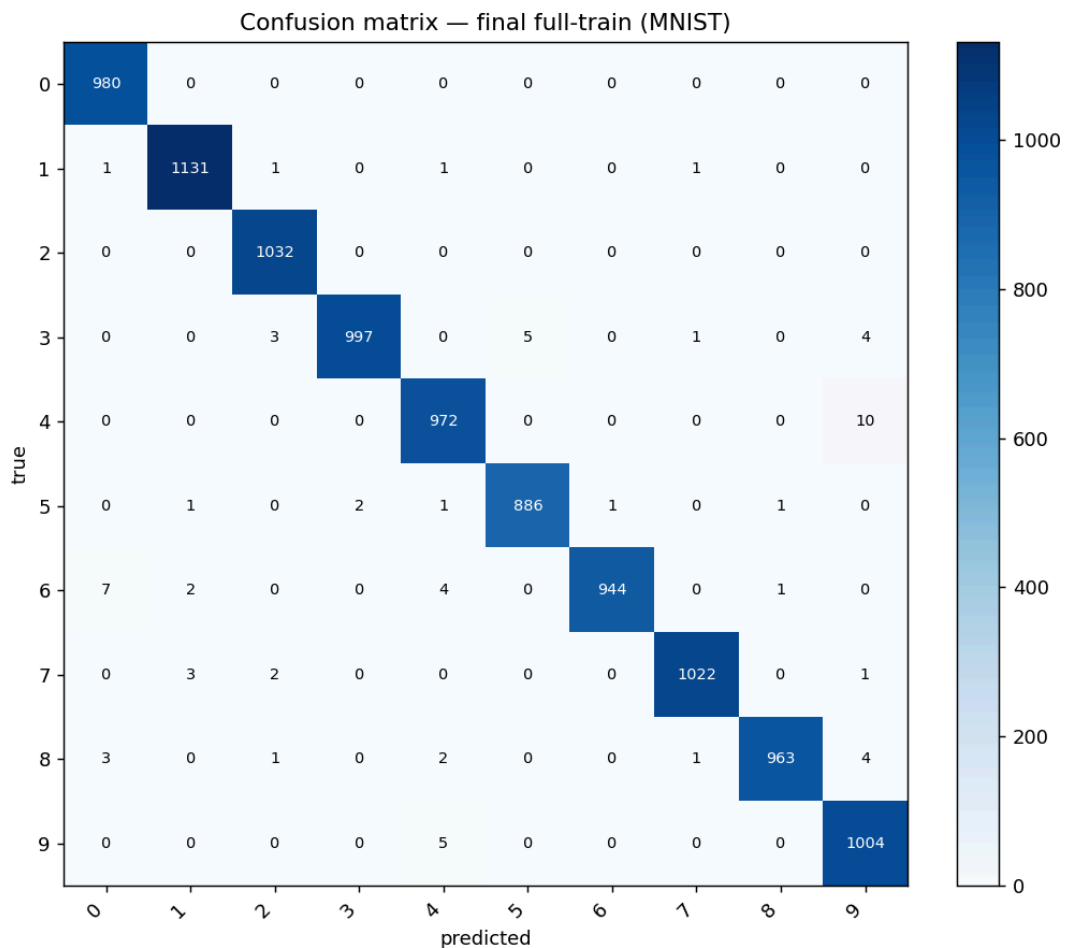


Рис. 5. Confusion matrix лучшей модели на test после финального обучения на полном train.

Матрица почти диагональная — это характерный признак очень хорошей модели на MNIST. Отдельные ошибки сосредоточены в парах визуально близких цифр (4 / 9, 5 / 3, 7 / 2), но даже там число ошибок исчисляется единицами на 1000 примеров.

4.6.4. Влияние отдельных гиперпараметров

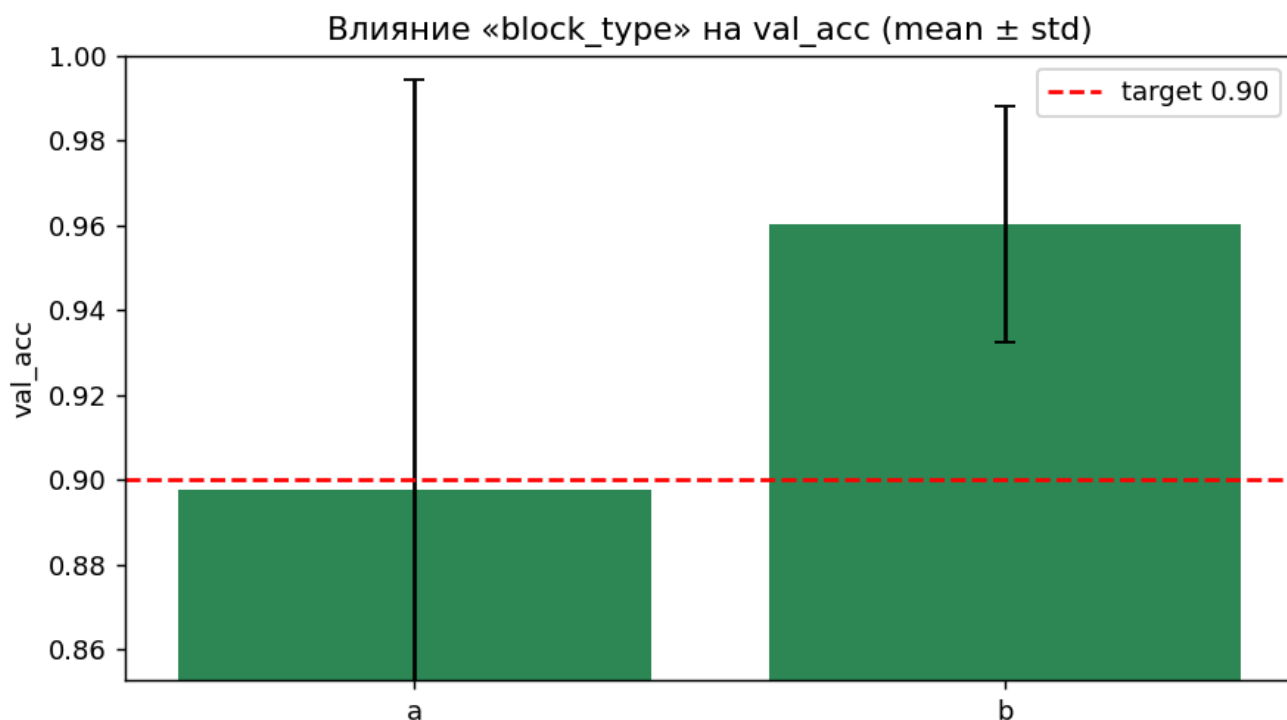


Рис. 6. Влияние типа блока (A vs B) на средний val_acc.

Блок В (две свёртки внутри блока) даёт стабильно лучшие результаты, чем А. Разрыв средних значений небольшой, но устойчивый, что подтверждает: **более глубокий блок извлекает более выразительные локальные признаки.**

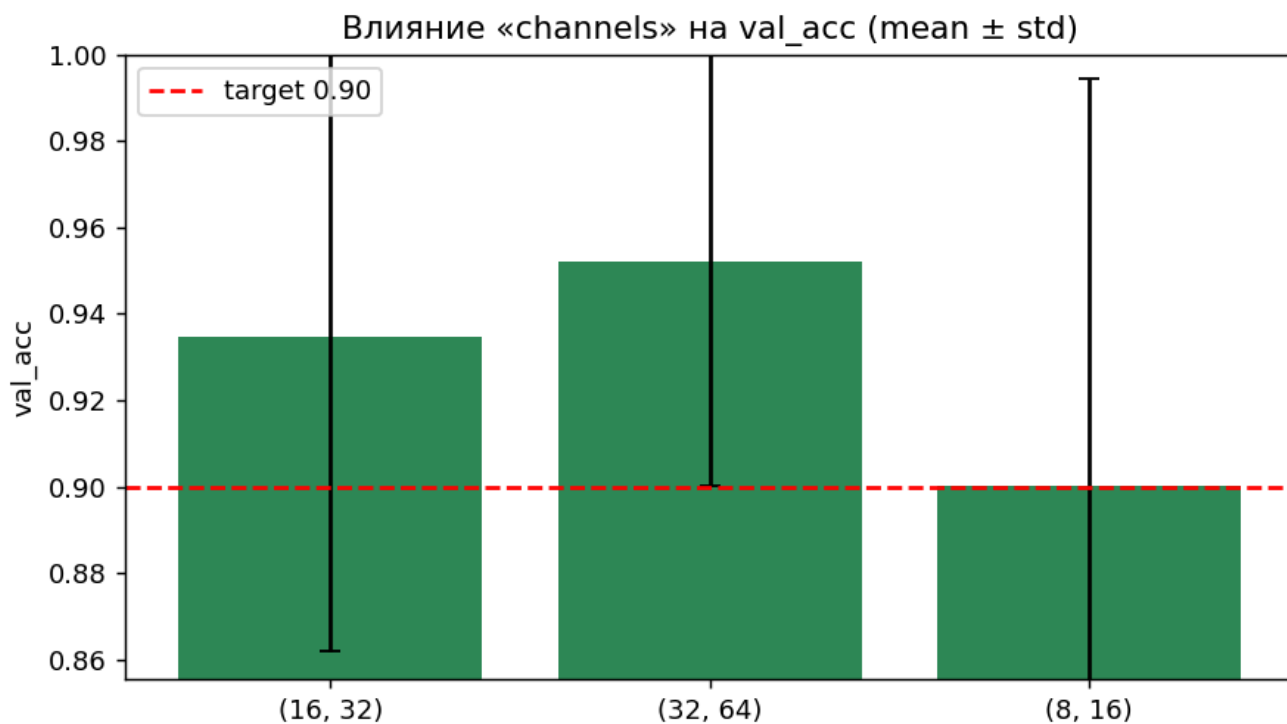


Рис. 7. Влияние числа каналов на val_acc.

Чётко прослеживается монотонная зависимость: $(8, 16) < (16, 32) < (32, 64)$. Больше число каналов — больше параллельных «фильтров-детекторов» — больше выразительность.

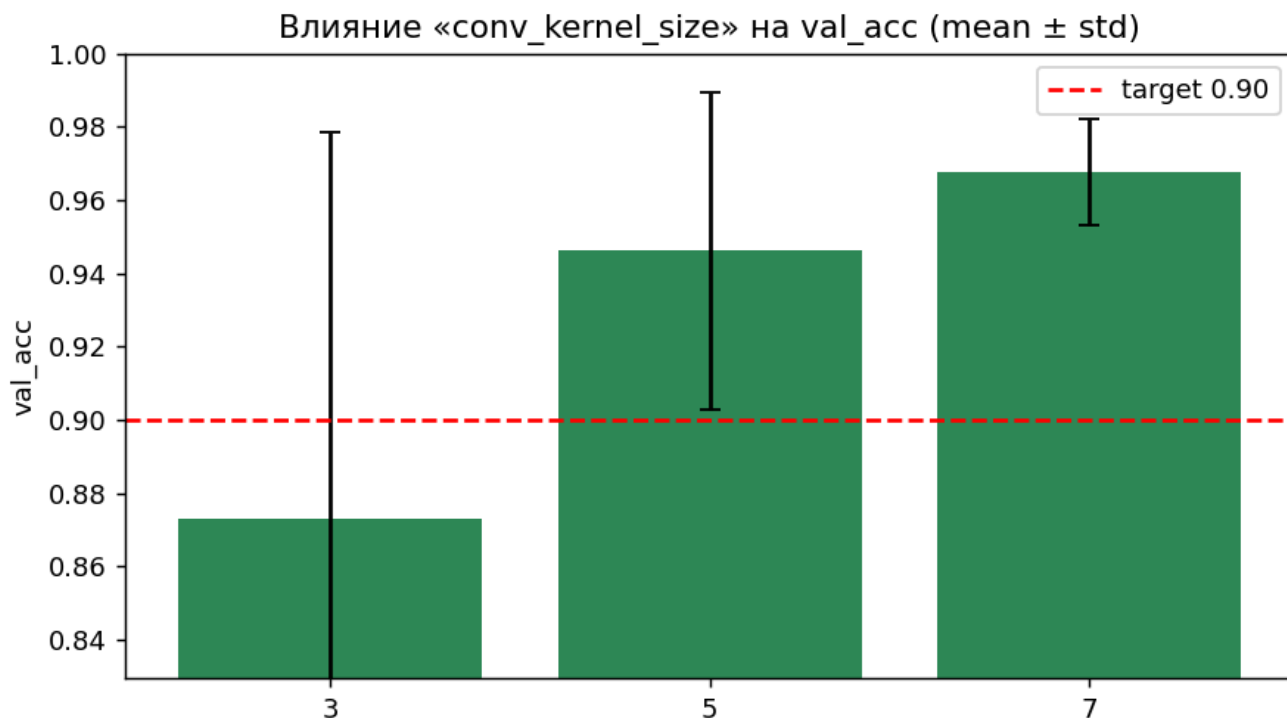


Рис. 8. Влияние размера ядра свёртки.

Размер ядра 5 и 7 в среднем заметно лучше, чем 3. Для MNIST с её крупными штрихами цифр большее ядро видит более информативный контекст за один шаг свёртки.

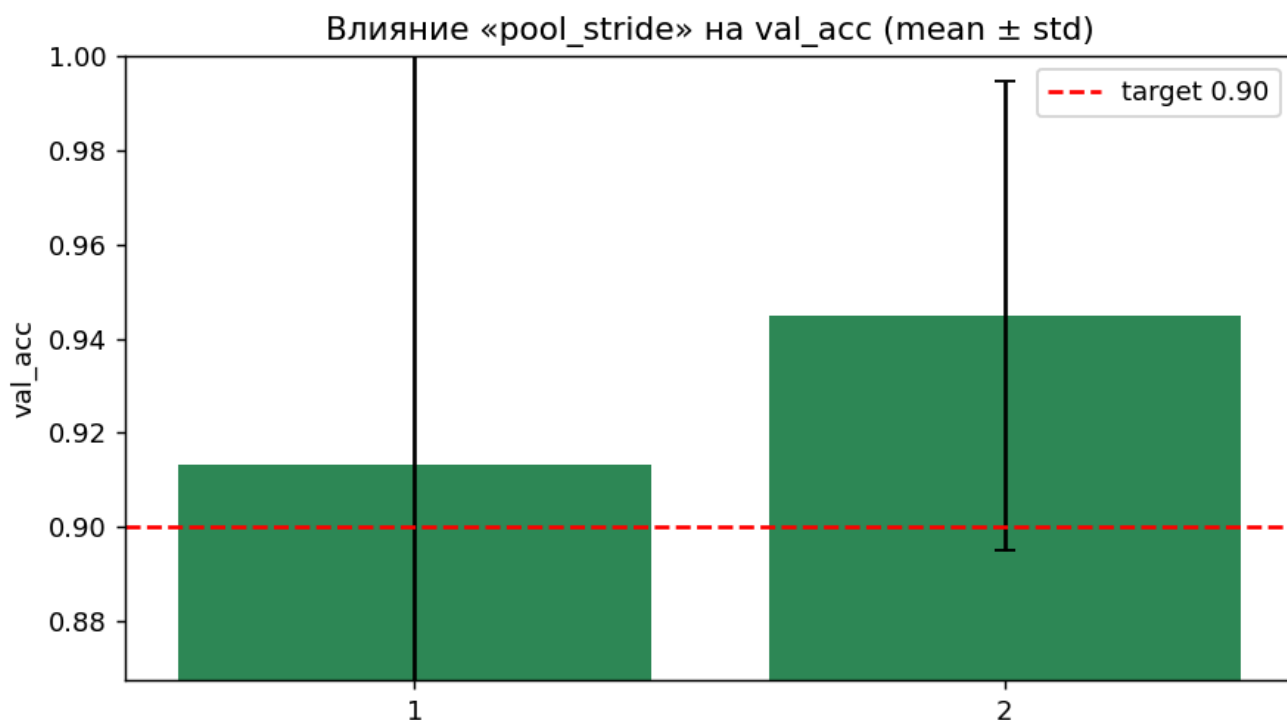


Рис. 9. Влияние шага pooling на val_acc.

`pool_stride = 2` обычно лучше, чем `pool_stride = 1`: более агрессивное уменьшение пространства снижает число параметров финального классификатора и заставляет фильтры выделять действительно сильные признаки.

4.7. Блок 2: оптимизаторы

Базовая архитектура: `block=B`, `channels=(32, 64)`, `conv_k=3`. Сравнение пяти вариантов оптимизатора и `lr` за 10 эпох на 8000 train-примеров MNIST:

Оптимизатор	lr	best val_acc	ep@90
Adam	0.0010	0.9405	6
Adam	0.0030	0.9685	4
AdamW	0.0010	0.9390	6
SGD	0.0100	0.9360	10
SGD	0.0500	0.8550	—

Таблица 2. Сравнение оптимизаторов на MNIST (10 эпох).

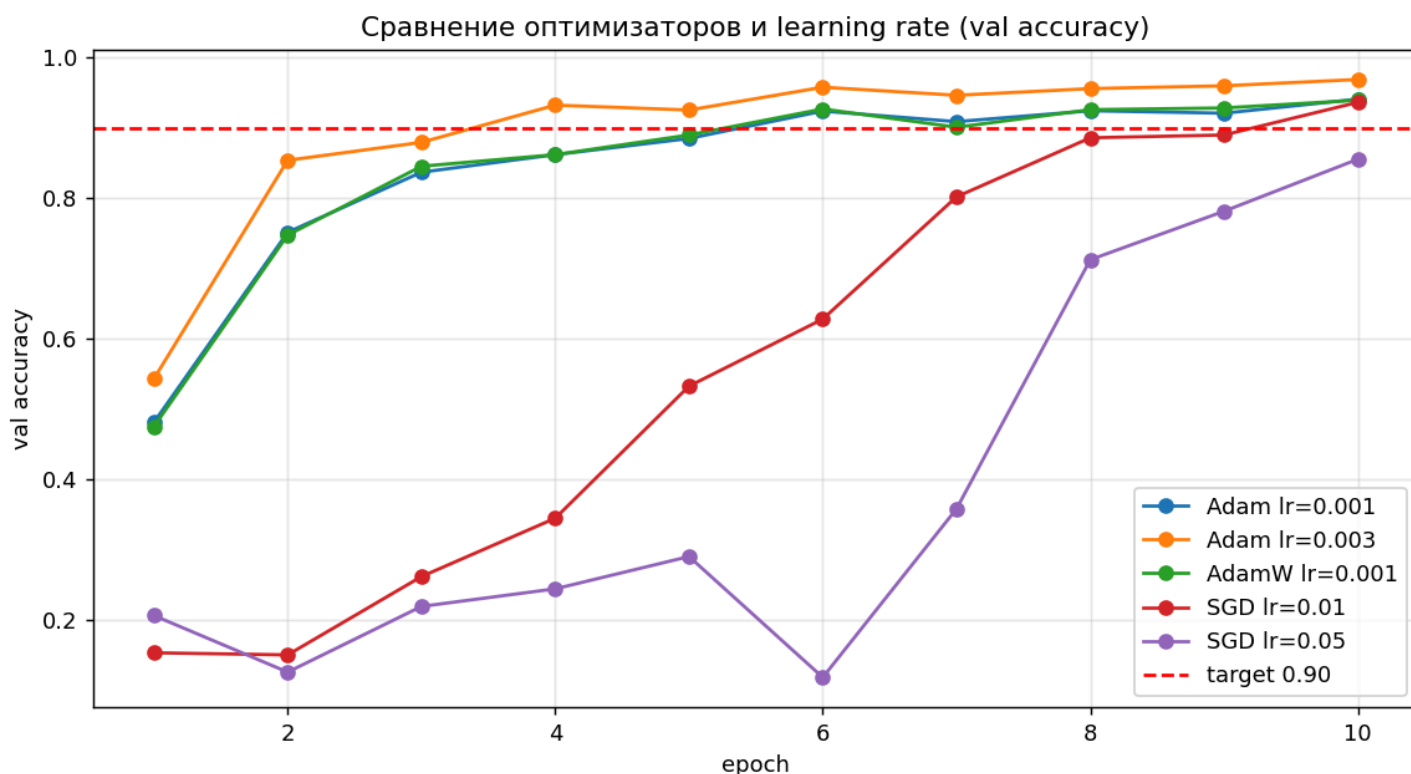


Рис. 10. Кривые `val_acc` для пяти комбинаций оптимизатора и `lr` на MNIST.

Видно, что:

- **Adam** со стандартным `lr = 1e-3` — надёжный «дефолт»;
- **Adam** с `lr = 3e-3` за то же число эпох выходит выше всех — адаптивный шаг хорошо справляется на этой задаче;
- **AdamW** почти неотличим от **Adam** при таком малом числе эпох — `weight decay` здесь не критичен;
- **SGD** стартует медленно и при `lr = 0.05` ведёт себя нестабильно (на эпохе 6 виден резкий выброс `val_loss`), что характерно для слишком большого шага.

4.8. Блок 2: переобучение и Dropout

На маленькой подвыборке (240 train, 60 val) сравниваются два режима той же архитектуры: без регуляризации (`dropout=0`) и с `dropout=0.4`. Длина обучения — 120 эпох без ранней остановки.

Режим	train_acc	val_acc	gap
без регуляризации	0.9542	0.8500	+0.1042
<code>dropout = 0.4</code>	0.7792	0.8833	−0.1042

Таблица 3. Итоги эксперимента на переобучение (MNIST).

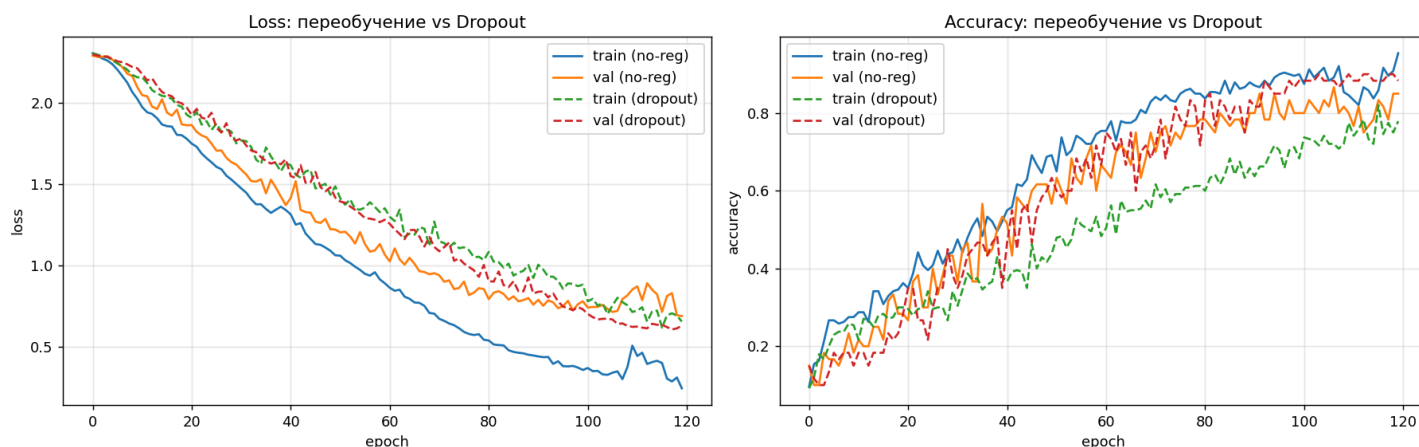


Рис. 11. Loss и accuracy для двух режимов: без регуляризации и с `Dropout = 0.4` (MNIST, 240 train).

На графиках видно:

- **без регуляризации** train loss падает почти до нуля, а val loss перестаёт улучшаться и начинает медленно расти — это классический признак переобучения; зазор `train - val` по accuracy раздувается до $\approx 10\%$;
- **с `Dropout 0.4`** train accuracy ниже, но `val_acc` оказывается выше, чем без регуляризации; gap отрицательный, т. е. модель на val работает лучше, чем на train — что означает, что обучение ещё не «переедено».

Это хорошая иллюстрация того, что Dropout не просто «ухудшает обучение», а перенаправляет ёмкость модели в более устойчивые представления.

5. Часть II. Эксперимент на FashionMNIST

FashionMNIST имеет такой же формат (28×28 , 10 классов), но содержит изображения предметов одежды и обуви. Классы визуально ближе друг к другу (например, `Shirt / T-shirt / Coat / Pullover`), поэтому задача заметно сложнее.

5.1. Превью датасета



Рис. 12. Случайные изображения из FashionMNIST. Сразу видно визуальное сходство классов — особенно верхней одежды.

5.2. Параметры эксперимента

Сетка для FashionMNIST взята уже, чем для MNIST (16 валидных конфигураций), а число эпох увеличено до 40, поскольку задача сложнее:

- блоки: A, B;
- каналы: (16, 32), (32, 64);
- размер ядра: 3, 5;
- шаг pooling: 1, 2;
- learning rate: 0.001;
- эпох: до 40, ранняя остановка по `val_acc`.

5.3. Топ конфигураций по `val_acc`

#	блок	каналы	ck	ps	best val_acc	epochs
15	B	(32, 64)	5	1	0.8870	40
16	B	(32, 64)	5	2	0.8805	23
12	B	(16, 32)	5	2	0.8695	40
11	B	(16, 32)	5	1	0.8550	40
14	B	(32, 64)	3	2	0.8415	33
10	B	(16, 32)	3	2	0.8315	38
13	B	(32, 64)	3	1	0.8315	40
8	A	(32, 64)	5	2	0.8280	33

Таблица 4. Топ-8 конфигураций для FashionMNIST.

Здесь ни одна конфигурация не достигла `val_acc >= 0.90` за 40 эпох на 8000 train — задача действительно сложнее.

5.4. Лучшая конфигурация

```
block=B | channels=(32, 64) | conv(k=5, s=1) | pool(k=2, s=1)
best val_acc: 0.8870          лучшая эпоха: 38
параметров: 180 842          время обучения: 106.0 сек
test loss: 0.4262            test accuracy: 0.8605
```

После финального дообучения на полном train (60 000 объектов, 40 эпох):

```
final test loss: 0.2551
final test accuracy: 0.9248
```

5.5. Графики и их интерпретация

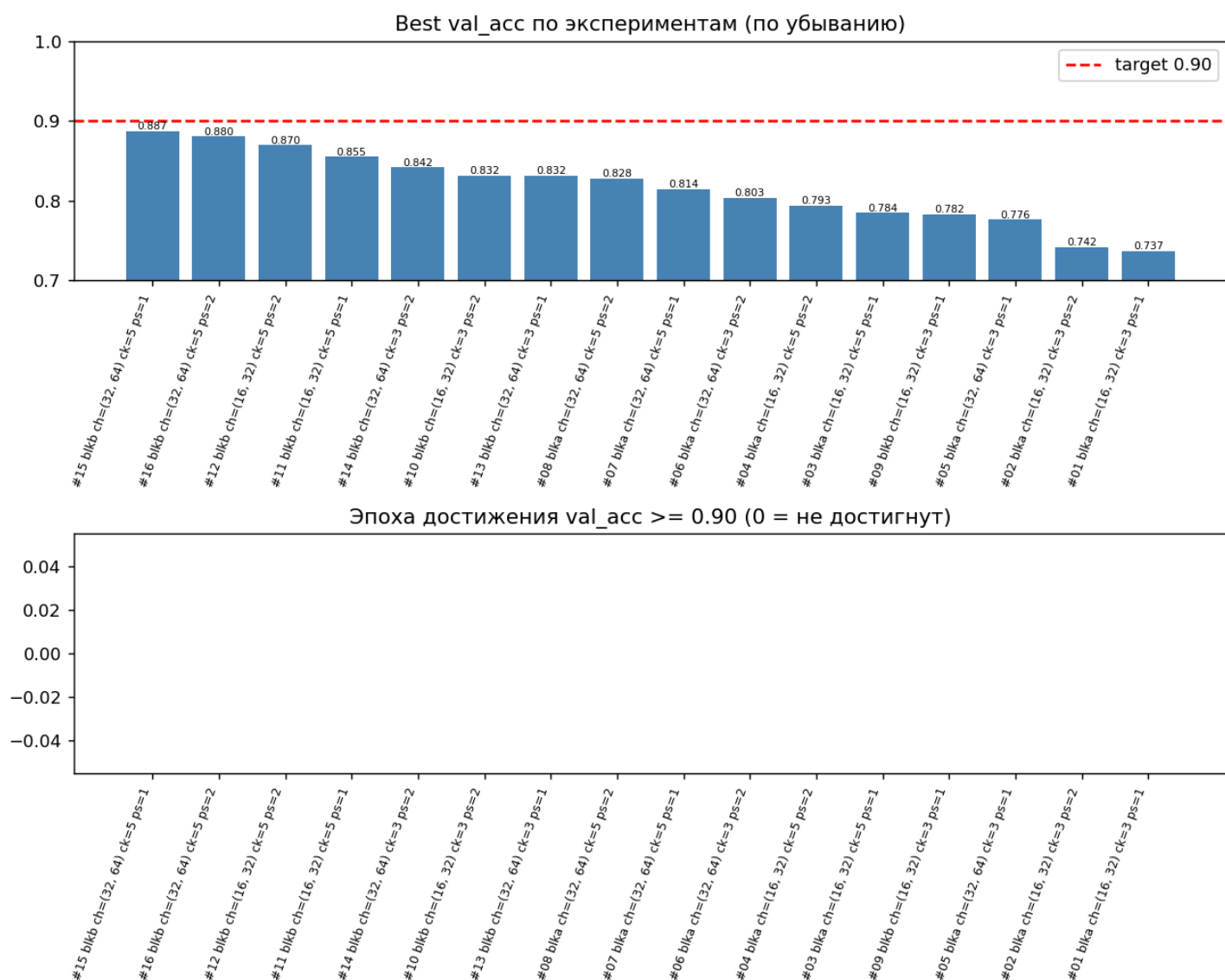


Рис. 13. Сводный график: best val_acc (слева) и эпохи до достижения 90% (справа) для FashionMNIST.

В отличие от MNIST, правый график пустой почти у всех конфигураций: **цели 90% за 40 эпох никто не достиг**. Это сразу визуализирует усложнение задачи.

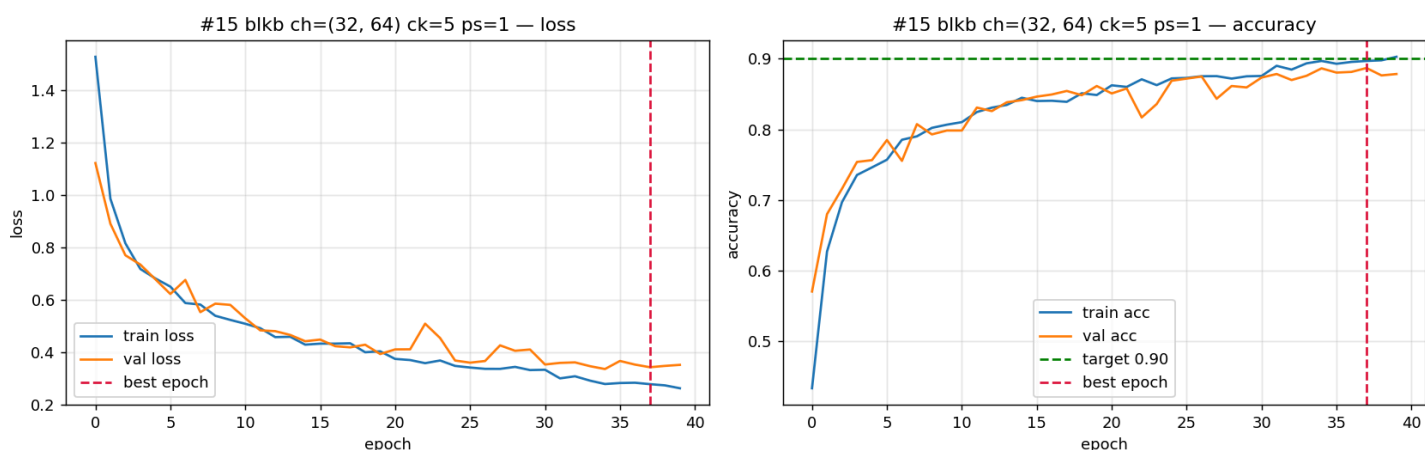


Рис. 14. Loss и accuracy лучшей конфигурации на FashionMNIST.

Кривые ведут себя «зашумлённое», чем на MNIST: видны небольшие скачки `val_loss`, `train` идёт устойчиво вниз, `val` выходит на плато около 0.88. Это говорит о том, что модель уже близка к пределу при таком объёме данных.

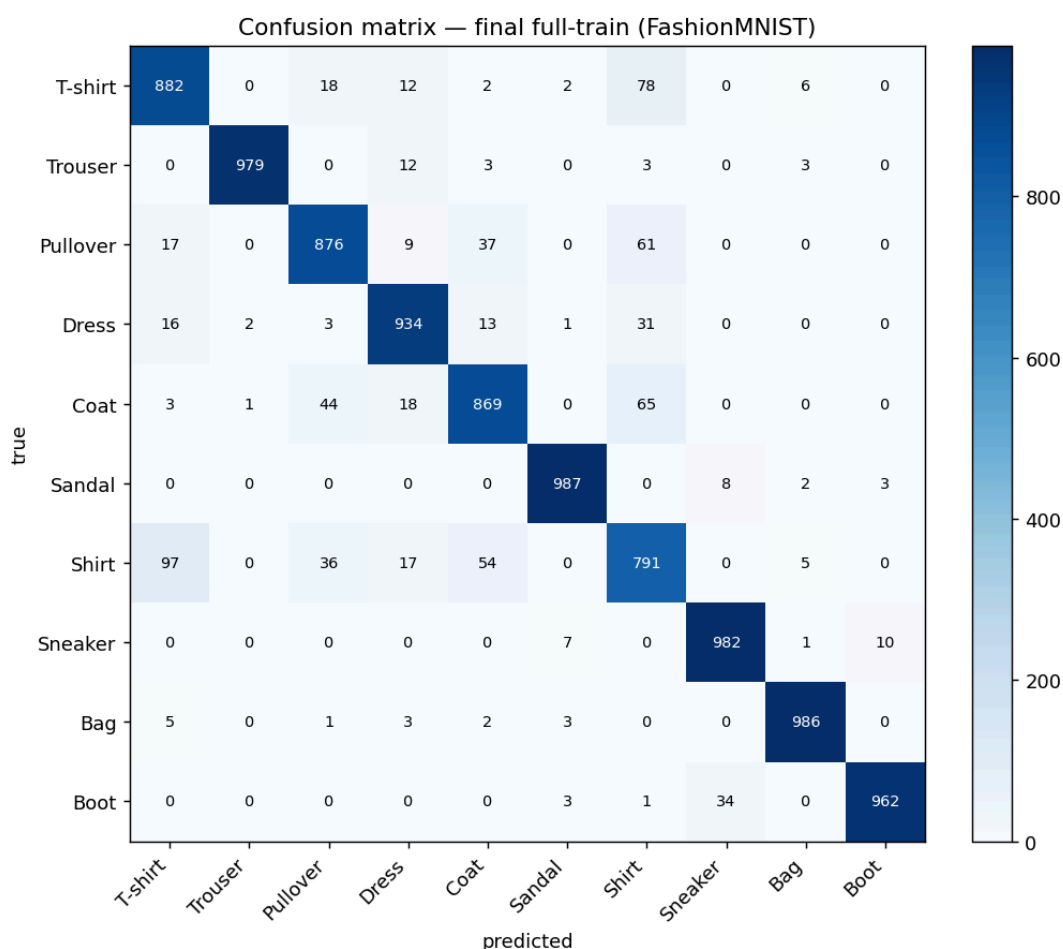


Рис. 15. Confusion matrix лучшей модели на test FashionMNIST.

В отличие от MNIST, матрица не идеально диагональная. Чётко видны **устойчивые ошибки между визуально похожими классами одежды**: Shirt -- T-shirt, Shirt -- Pullover -- Coat, Sneaker -- Ankle boot. Это содержательный, а не «случайный» паттерн ошибок: модель путает то, что и человек путает на 28×28 изображениях.

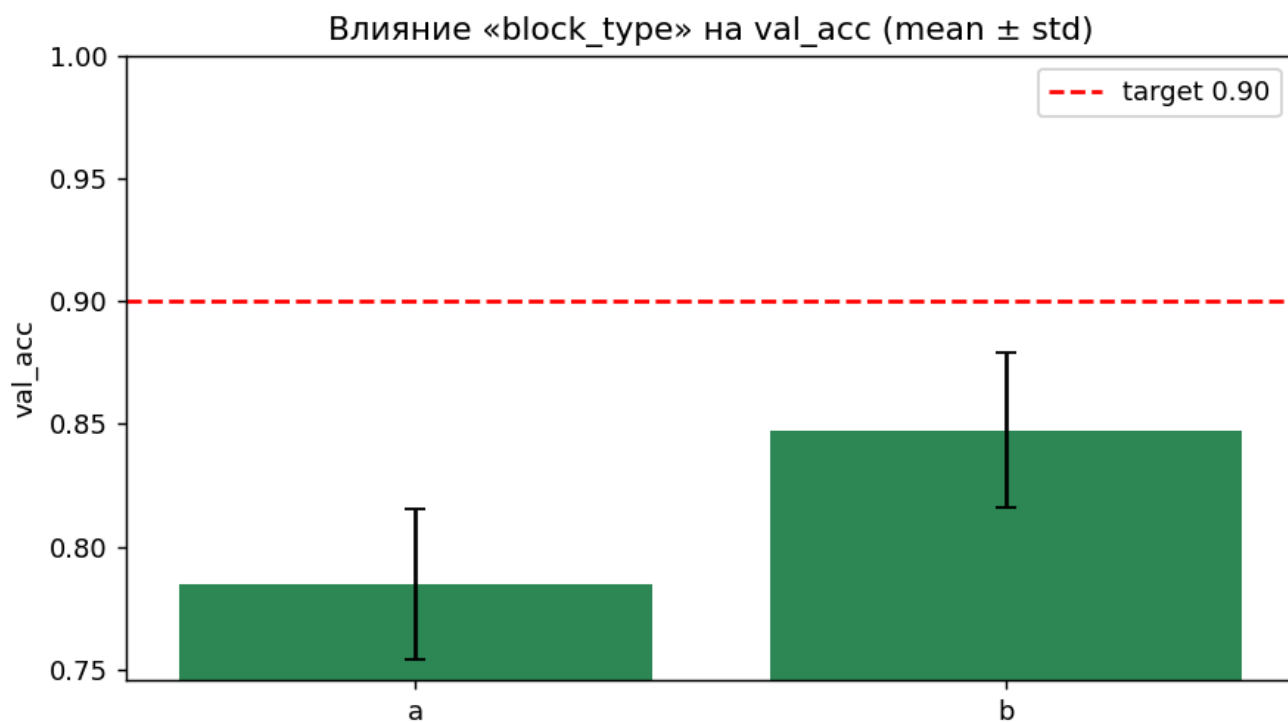


Рис. 16. Влияние типа блока (A vs B) на FashionMNIST.

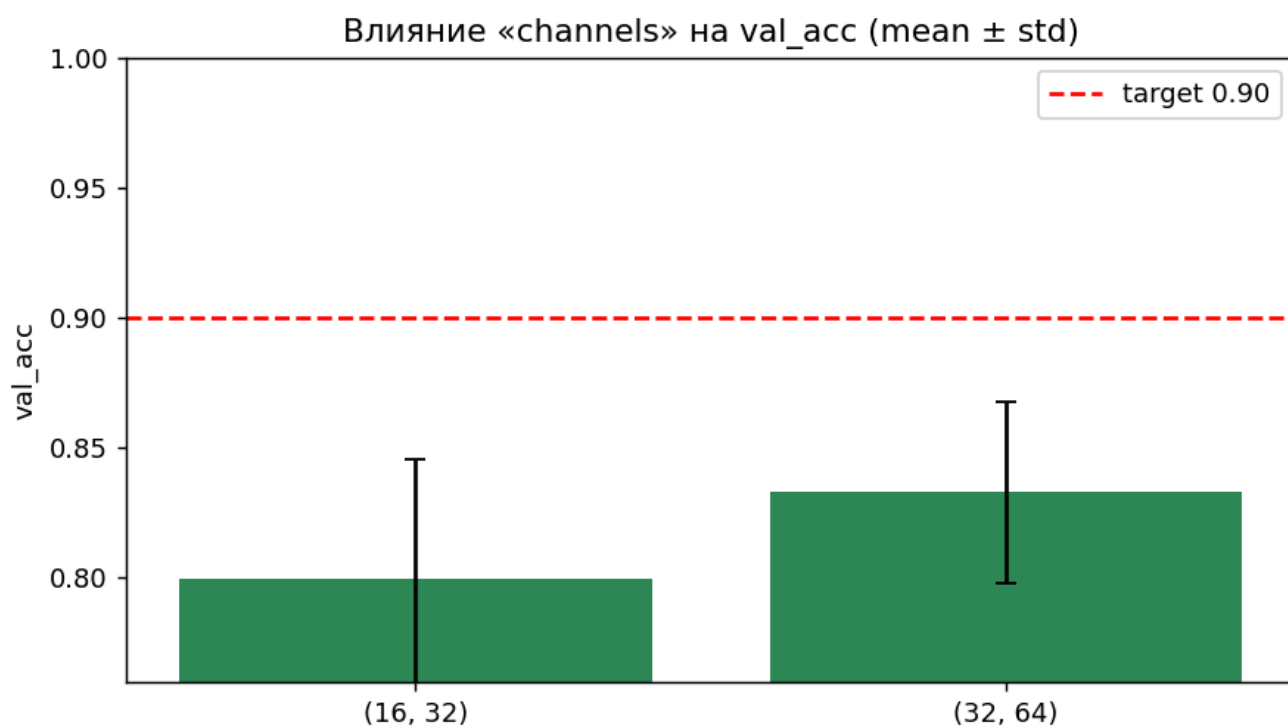


Рис. 17. Влияние числа каналов на FashionMNIST.

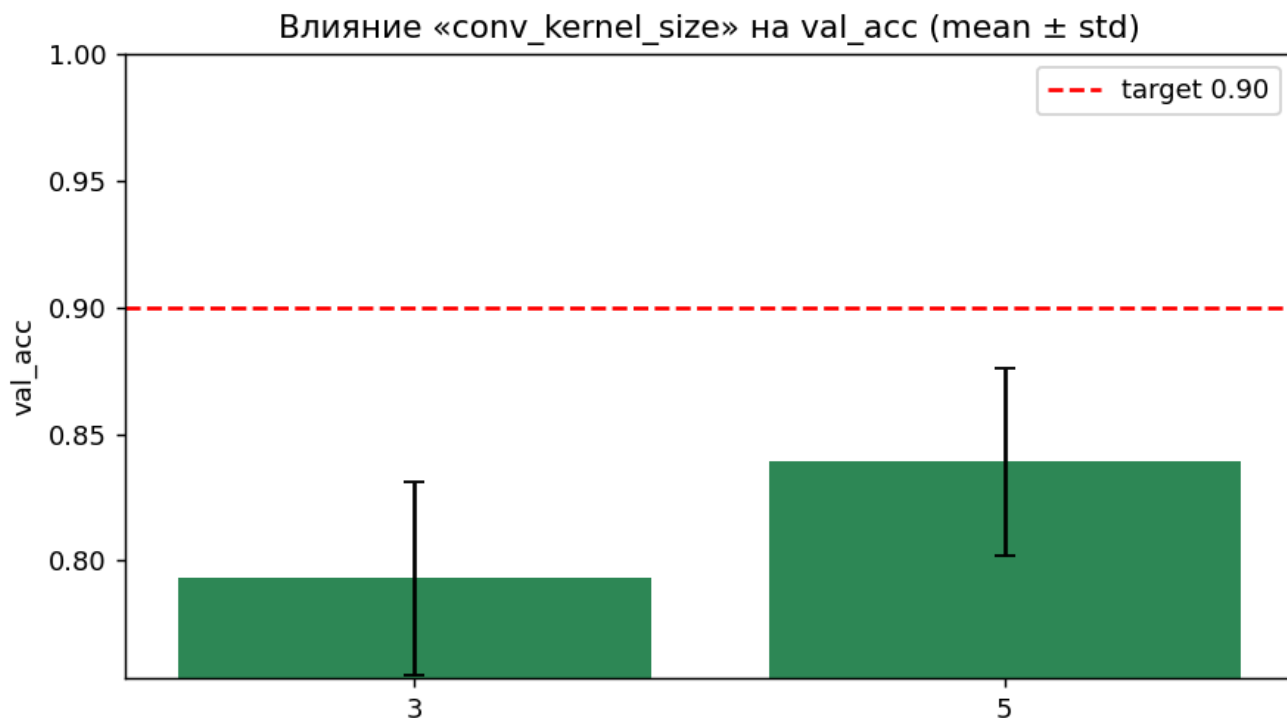


Рис. 18. Влияние размера ядра свёртки на FashionMNIST.

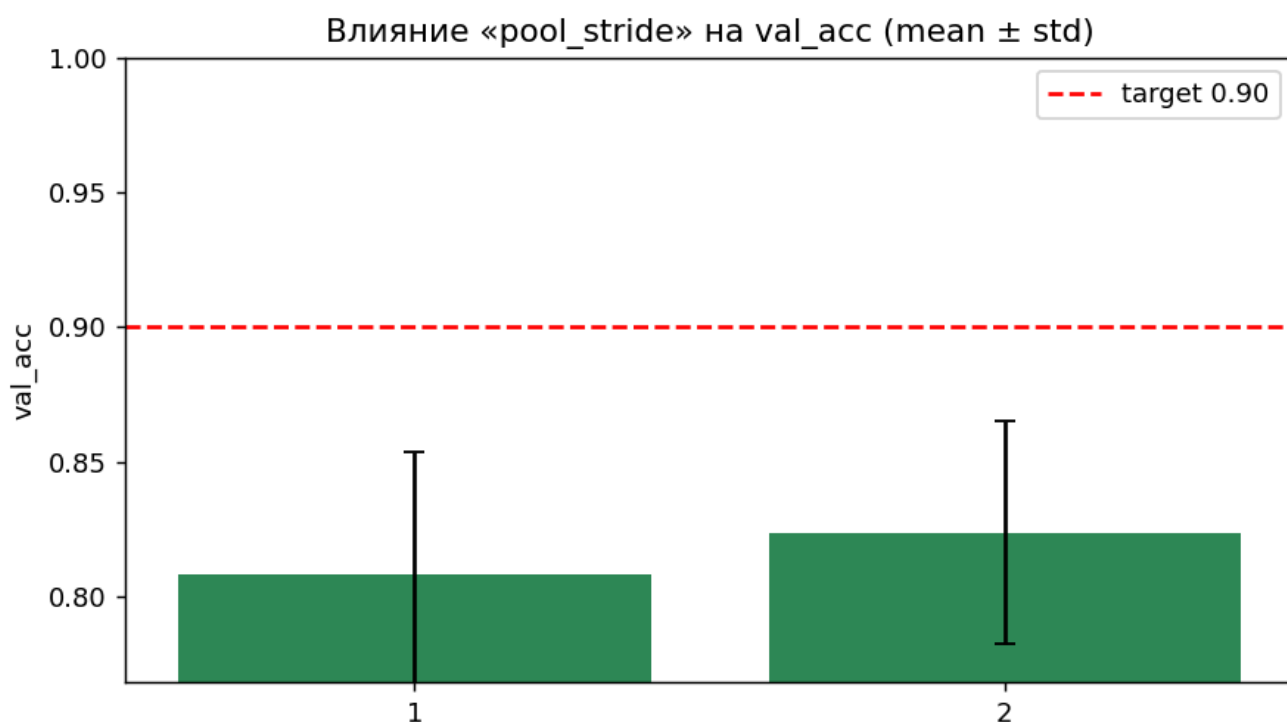


Рис. 19. Влияние шага pooling на FashionMNIST.

На FashionMNIST все четыре зависимости подтверждаются и выражены даже сильнее, чем на MNIST:

- блок В уверенно обходит А;
- (32, 64) заметно лучше (16, 32);
- ядро 5 лучше ядра 3 (фактуры одежды требуют большего receptive field);
- больший pool_stride улучшает качество.

5.6. Блок 2: оптимизаторы

Оптимизатор	lr	best val_acc	ep@90
Adam	0.0010	0.7790	—
Adam	0.0030	0.8330	—
AdamW	0.0010	0.7810	—
SGD	0.0100	0.7350	—
SGD	0.0500	0.6840	—

Таблица 5. Сравнение оптимизаторов на FashionMNIST (10 эпох).

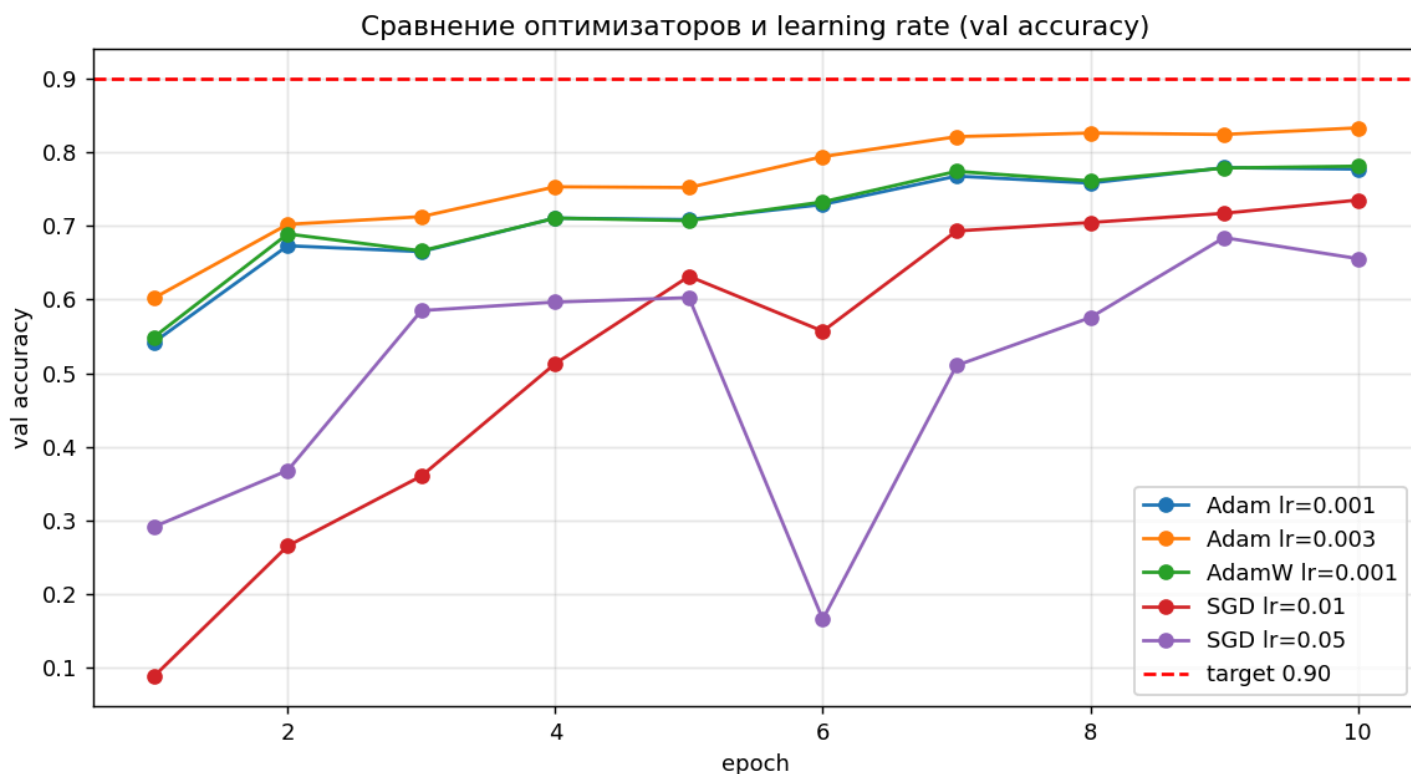


Рис. 20. Сравнение оптимизаторов на FashionMNIST.

Картина та же, что и на MNIST: победитель — Adam с $lr = 3e-3$, AdamW $\approx$ Adam, SGD отстаёт. Однако абсолютные значения существенно ниже, и достижение $acc = 90\%$ за 10 эпох недостижимо.

5.7. Блок 2: переобучение и Dropout

Режим	train_acc	val_acc	gap
без регуляризации	0.8292	0.6667	+0.1625
dropout = 0.4	0.7458	0.6667	+0.0792

Таблица 6. Итоги эксперимента на переобучение (FashionMNIST).

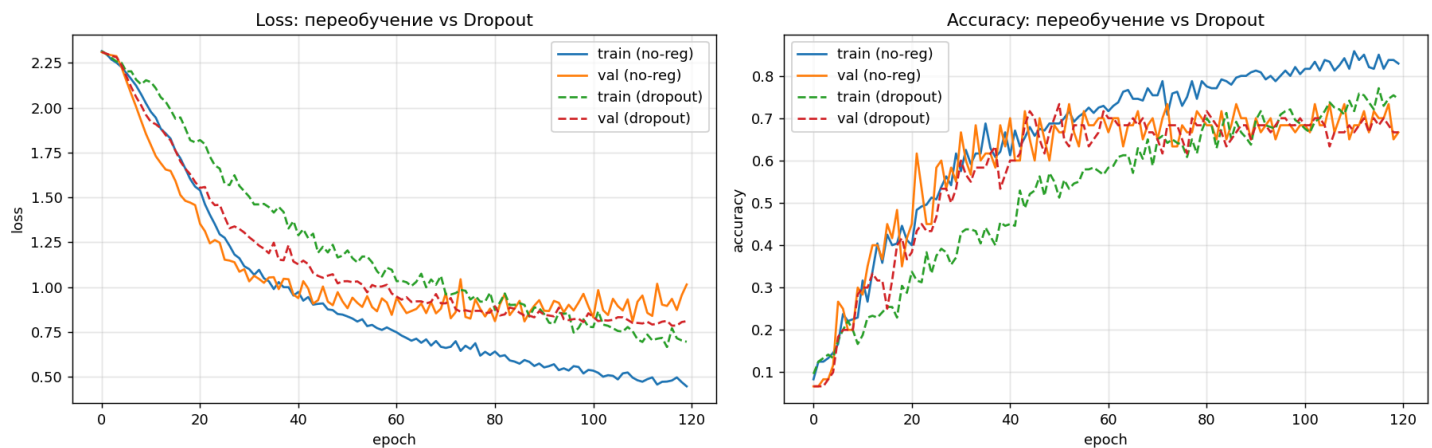


Рис. 21. Loss и accuracy: переобучение vs Dropout на FashionMNIST.

На FashionMNIST переобучение проявляется ещё ярче, чем на MNIST:

- без регуляризации `val_loss` стабильно растёт почти всё обучение, а `train_loss` падает — классическая картина переобучения;
- с `Dropout = 0.4` `val_acc` тот же (0.6667), но **gap почти вдвое меньше**, и `val_loss` ведёт себя устойчивее.

То есть Dropout не вытащил `val`-метрику выше (240 объектов — слишком мало для сложной задачи), но **устранил расхождение между `train` и `val`** — именно то, для чего он и нужен.

6. Сравнение MNIST и FashionMNIST

Метрика	MNIST	FashionMNIST
Лучший блок	B	B
Лучшие каналы	(32, 64)	(32, 64)
Лучшее ядро	7	5
Лучший <code>val_acc</code> (8k)	0.9875	0.8870
Test acc (8k train)	0.9835	0.8605
Test acc (full train)	0.9931	0.9248
Эпох до 90% (best)	2	—
Конфигураций $\geq 90\%$	59 / 72	0 / 16

Таблица 7. Сводное сравнение MNIST и FashionMNIST.

Тот же класс архитектур работает на обоих датасетах, но **разница в качестве почти 7 п. п.** объясняется природой данных: рукописные цифры легко отделимы по простым штрихам, а одежда требует более тонких текстурных признаков и большего объёма обучающих данных.

7. Выводы

1. Свёрточная сеть `ImageCNN`, реализованная средствами `torch.nn`, успешно решает задачу классификации на обоих датасетах: 0.9931 test accuracy на MNIST и 0.9248 на FashionMNIST.
2. Систематический перебор гиперпараметров показал стабильно работающий рецепт: **блок В, каналы (32, 64), крупное ядро свёртки, `pool_stride = 2`**.
3. Глубокий блок В (две свёртки до pooling) уверенно лучше простого А на обоих датасетах. Прирост числа каналов стабильно улучшает качество.
4. Размер ядра свёртки имеет значение: на MNIST лучшим оказалось ядро 7×7 , на FashionMNIST — 5×5 .
5. MNIST — лёгкая задача: 59 из 72 конфигураций достигли `acc = 90%`, причём лучшие — за 2 эпохи. FashionMNIST на тех же 8000 train не пробил порог 90% ни в одной конфигурации, но при обучении на полном train поднялся до 92.5%.
6. Среди оптимизаторов лучший результат за 10 эпох даёт **Adam с `lr = 3e-3`**. AdamW почти неотличим от Adam. SGD требует более аккуратной настройки и большего числа эпох; при `lr = 0.05` он становится неустойчивым.
7. Эксперимент на маленькой подвыборке наглядно демонстрирует переобучение: train loss падает, val loss растёт, gap по accuracy раздувается. Dropout = 0.4 сокращает или вовсе устраняет этот gap, делая обучение более устойчивым.
8. Confusion matrices показывают содержательную природу ошибок: на MNIST ошибки сосредоточены на визуально близких цифрах (4 / 9, 5 / 3), на FashionMNIST — на семантически близких классах одежды (Shirt / T-shirt / Coat).

8. Структура проекта

```
SMGM0/
├─ lab4/
│   ├── main.typ          # отчёт (Typst)
│   ├── config.py         # параметры через env-переменные
│   ├── data.py           # загрузка MNIST/FashionMNIST, DataLoader
│   ├── cnn.py            # ImageCNN, обучение, оптимизаторы
│   ├── block1_main.py    # Блок 1: перебор архитектур
│   ├── block2_main.py    # Блок 2: оптимизаторы и Dropout
│   ├── gui_launcher.py   # GUI-лаунчер (Tkinter)
│   ├── results-MNIST/    # результаты по MNIST
│   └─ results-FashionMNIST/ # результаты по FashionMNIST
```


9. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>